



Analizador de Testes A/B de Cashback

Descrição da arquitetura e das responsabilidades dos scripts

Case técnico realizado por

Pedro Moreira Beiro

Sumário

1	Visão geral do projeto	2
2	Fluxo geral da aplicação	2
3	Descrição dos scripts	3
3.1	app.py	3
3.2	data_preparation_validation.py	3
3.3	descriptive_ranking_analyzer.py	5
3.4	warning_analyzer.py	5
3.5	report_generator.py	5
3.6	results_logger.py	6
3.7	google_sheets_logger.py	6
3.8	llm_client.py	7
3.9	natural_language_response.py	7
3.10	integracao.py	8
3.11	__init__.py	8
4	Organização da arquitetura	8
5	Tratamento de falhas	9
6	Possíveis melhorias futuras	9

1 Visão geral do projeto

O projeto consiste em uma aplicação para análise de testes A/B de cashback. A solução recebe uma solicitação em linguagem natural, identifica o tipo de análise desejado, processa os dados de um arquivo CSV, compara os grupos avaliados e apresenta os resultados em uma interface web.

A aplicação oferece dois modos principais de análise. O modo de ranking compara os grupos de acordo com uma métrica e um objetivo de maximização ou minimização. O modo de relatório geral apresenta uma visão consolidada do desempenho de todos os grupos, sem selecionar um vencedor.

Além da análise principal, a solução identifica possíveis conflitos entre a métrica solicitada e indicadores financeiros complementares previamente definidos (hard-coded). Esses conflitos são apresentados como pontos de atenção ou alertas críticos.

Ao final da execução, a aplicação gera um relatório HTML, registra o resultado em um histórico CSV local e, quando configurado, sincroniza o mesmo registro com uma planilha do Google Sheets [Beiro 2026].

2 Fluxo geral da aplicação

O fluxo da solução pode ser resumido nas seguintes etapas:

1. recebimento da solicitação em linguagem natural;
2. interpretação da solicitação pelo modelo Gemini;
3. seleção do arquivo de dados;
4. leitura, validação e preparação do CSV;
5. execução do ranking ou do relatório geral;
6. análise de alertas financeiros;
7. geração do relatório HTML e dos gráficos;
8. registro local dos resultados;
9. sincronização opcional com o Google Sheets;
10. geração da resposta final em linguagem natural.

Cada script possui uma responsabilidade específica dentro desse fluxo. Essa separação reduz o acoplamento entre as partes da aplicação e facilita manutenção, testes e futuras extensões.

3 Descrição dos scripts

3.1 `app.py`

O arquivo `app.py` contém a interface principal da aplicação, desenvolvida com Streamlit. Ele é responsável por apresentar os componentes visuais e permitir a interação com o usuário.

A interface recebe a solicitação em linguagem natural e permite o envio opcional de um arquivo CSV. Também apresenta exemplos de perguntas, descrições dos modos de análise, objetivos e métricas disponíveis.

Durante a execução, o script mostra o andamento das tentativas de comunicação com o Gemini. Caso a interpretação automática não seja possível, a interface disponibiliza uma configuração manual, na qual o usuário escolhe o modo, o parceiro, a métrica e o objetivo.

Após a análise, o arquivo apresenta:

- resposta final em linguagem natural;
- grupo selecionado;
- métrica analisada;
- valor obtido;
- ranking dos grupos;
- alertas financeiros;
- status dos registros;
- botões para download dos arquivos;
- acesso ao histórico no Google Sheets;
- detalhes técnicos da execução.

O script também controla arquivos temporários enviados pelo usuário e possui uma opção para encerrar o servidor local.

3.2 `data_preparation_validation.py`

O arquivo `data_preparation_validation.py` concentra a etapa de preparação e validação dos dados.

Sua responsabilidade inicial é ler o arquivo CSV e verificar se ele possui a estrutura esperada. São avaliados aspectos como presença das colunas obrigatórias, consistência dos tipos de dados, valores ausentes e linhas inválidas.

Após a validação, o script realiza ajustes necessários para que os dados possam ser utilizados pelas etapas seguintes. Entre esses ajustes estão a conversão de datas, a transformação de colunas numéricas e a padronização dos nomes dos grupos e parceiros.

O script também calcula as métricas derivadas utilizadas nas análises, como:

- receita líquida:

$$Receitalíquida = Comissão - Cashback \quad (1)$$

- vendas por comprador:

$$Vendasporcomprador = \frac{Vendastotais}{Número decompradores} \quad (2)$$

- comissão por comprador:

$$Comissão por comprador = \frac{Comissão}{Número decompradores} \quad (3)$$

- cashback por comprador:

$$Cashback por comprador = \frac{Cashback}{Número decompradores} \quad (4)$$

- receita líquida por comprador:

$$Receitalíquida por comprador = \frac{Receitalíquida}{Número decompradores} \quad (5)$$

- taxa de comissão:

$$Taxa de comissão = \frac{Comissão}{Vendastotais} \quad (6)$$

- taxa de cashback:

$$Taxa de cashback = \frac{Cashback}{Vendastotais} \quad (7)$$

- margem líquida:

$$Margem líquida = \frac{Receitalíquida}{Vendastotais} \quad (8)$$

Além dos dados preparados, o módulo produz um relatório de validação contendo informações como número de linhas iniciais, linhas finais, linhas removidas, quantidade de datas avaliadas, avisos e erros encontrados.

3.3 `descriptive_ranking_analyzer.py`

O arquivo `descriptive_ranking_analyzer.py` executa a análise descritiva dos grupos.

No modo de ranking, o script consolida os dados por grupo, calcula os resultados da métrica selecionada e ordena os grupos conforme o objetivo definido. Para maximização, os maiores valores recebem as primeiras posições. Para minimização, os menores valores são priorizados.

A saída inclui o grupo selecionado, sua posição, o valor observado e a comparação com os demais grupos. Também são calculadas diferenças absolutas e percentuais entre o vencedor e seus concorrentes.

No modo de relatório geral, o script não seleciona um vencedor. Em vez disso, organiza todas as métricas consolidadas para cada grupo, permitindo uma visão comparativa mais ampla do teste.

Esse módulo realiza apenas cálculos determinísticos, sem participação do modelo de linguagem na escolha dos resultados.

3.4 `warning_analyzer.py`

O arquivo `warning_analyzer.py` analisa possíveis conflitos entre o resultado principal e indicadores financeiros complementares.

Um grupo pode apresentar o melhor valor na métrica solicitada, mas possuir desempenho inferior em receita líquida, margem líquida ou vendas totais. O objetivo desse módulo é identificar situações em que o vencedor do ranking pode não representar a melhor decisão de negócio.

Os alertas são divididos em níveis.

O ponto de atenção indica que o grupo selecionado apresenta uma defasagem relevante em alguma métrica financeira complementar. O alerta crítico representa uma condição mais grave, como receita líquida nula ou negativa.

Esses alertas não alteram o ranking. Eles são apresentados como informações adicionais para apoiar uma decisão mais segura antes de escalar uma variante.

3.5 `report_generator.py`

O arquivo `report_generator.py` gera os relatórios HTML e os gráficos da análise.

O relatório utiliza uma identidade visual baseada nas cores da Méliuz, com predominância de rosa-claro, rosa principal e branco. A logo é incorporada ao documento e o layout é organizado com aparência de apresentação executiva.

Para análises de ranking, o relatório apresenta:

- título relacionado ao parceiro e à métrica;

- resumo executivo;
- grupo selecionado;
- valor principal;
- ranking das variantes;
- diferenças entre os grupos;
- alertas;
- indicadores financeiros;
- gráficos comparativos.

Para relatórios gerais, são apresentados cards de resumo, gráficos das métricas principais e tabelas com os indicadores de todos os grupos.

As informações técnicas, como nome do arquivo analisado, quantidade de linhas e status da validação, são posicionadas no rodapé de forma discreta.

Os gráficos e a logo são convertidos para Base64 e incorporados diretamente ao HTML. Dessa forma, o relatório pode ser movido ou enviado sem depender de arquivos externos.

3.6 `results_logger.py`

O arquivo `results_logger.py` registra os resultados das análises em um arquivo CSV local.

Cada nova execução adiciona uma linha ao histórico de acompanhamento. O registro reúne informações como data, parceiro, arquivo analisado, tipo de consulta, descrição, resultado, decisão, quantidade de alertas e mensagens associadas.

O histórico local funciona como uma forma de auditoria das análises realizadas. Ele também garante que os resultados sejam armazenados mesmo quando a sincronização com serviços externos não estiver disponível.

O arquivo CSV é criado automaticamente na pasta de saída e pode ser baixado pela interface Streamlit.

3.7 `google_sheets_logger.py`

O arquivo `google_sheets_logger.py` realiza a sincronização dos registros com o Google Sheets.

A comunicação é feita por meio de um webhook criado no Google Apps Script. O script envia uma requisição HTTP contendo o mesmo registro armazenado no histórico CSV local.

A integração utiliza uma URL e um token configurados por variáveis de ambiente. O token impede que requisições não autorizadas adicionem dados à planilha.

Caso o Google Sheets esteja indisponível, o token esteja incorreto ou ocorra um erro de conexão, a análise principal não é interrompida. O resultado continua sendo salvo localmente e a interface apenas informa que a sincronização não foi concluída.

O módulo retorna informações como status da sincronização, mensagem, código HTTP, aba utilizada e linha inserida.

3.8 `llm_client.py`

O arquivo `llm_client.py` concentra a comunicação com o modelo Gemini para interpretação das solicitações.

O modelo recebe a pergunta do usuário e identifica os parâmetros necessários para a análise, como:

- modo de análise;
- métrica;
- objetivo;
- parceiro;
- arquivo citado.

A resposta do modelo é convertida para uma estrutura validada. Isso evita que informações incompletas ou fora do padrão sejam enviadas diretamente para os módulos de cálculo.

O script também localiza o arquivo correspondente ao parceiro na pasta de dados quando nenhum CSV é carregado pelo usuário.

Para lidar com indisponibilidades temporárias, são realizadas até três tentativas de comunicação. O tempo de espera aumenta entre as tentativas, principalmente em casos de limite de requisições.

O progresso das tentativas pode ser enviado para a interface Streamlit, permitindo que o usuário acompanhe a execução.

3.9 `natural_language_response.py`

O arquivo `natural_language_response.py` transforma os resultados calculados em uma resposta curta e clara para o usuário.

Após a conclusão da análise, o módulo envia ao Gemini somente os dados necessários para redigir a explicação. O modelo não realiza novos cálculos e não pode alterar o grupo selecionado, os valores ou os alertas identificados pelo Python.

A resposta final busca apresentar o resultado de forma mais natural, destacando a métrica, o grupo escolhido, o valor observado e possíveis pontos de atenção.

Caso essa segunda comunicação com o Gemini falhe, o script gera uma resposta local determinística. Dessa forma, a análise continua apresentando uma conclusão mesmo quando o serviço externo está temporariamente indisponível.

3.10 `integracao.py`

O arquivo `integracao.py` coordena o fluxo completo da aplicação.

Ele conecta os módulos de interpretação, preparação dos dados, análise, alertas, relatórios, registros e resposta final.

A partir da solicitação estruturada, esse script seleciona o arquivo, prepara os dados, executa o modo solicitado e encaminha os resultados para as etapas posteriores.

No modo de ranking, também executa a análise de alertas. No modo de relatório geral, organiza os dados consolidados sem selecionar um vencedor.

Após os cálculos, o módulo solicita a geração do relatório HTML, registra o resultado no CSV local, tenta sincronizá-lo com o Google Sheets e produz a resposta final.

Esse arquivo funciona como a camada central da aplicação, mantendo a interface separada das regras de negócio.

3.11 `__init__.py`

O arquivo `__init__.py` identifica a pasta `scripts` como um pacote Python.

Isso permite que os módulos sejam importados de forma organizada, mantendo uma estrutura consistente entre a execução local, o Streamlit e os testes.

O arquivo não possui regras de negócio e serve apenas para a organização interna do projeto.

4 Organização da arquitetura

A arquitetura foi dividida em módulos com responsabilidades específicas.

A interface é mantida separada da análise dos dados. A comunicação com o Gemini também é isolada dos cálculos. Os registros locais e remotos utilizam módulos independentes, e a geração dos relatórios não interfere na lógica do ranking.

Essa estrutura permite que cada parte seja alterada ou testada sem exigir grandes mudanças nas demais.

Por exemplo, a indisponibilidade do Google Sheets não impede a análise, pois o histórico local permanece ativo. Da mesma forma, uma falha na resposta final do

Gemini não altera os cálculos, já que os resultados são produzidos anteriormente pelo Python.

5 Tratamento de falhas

A solução foi projetada para evitar que falhas em serviços externos interrompam todo o fluxo.

Os principais mecanismos são:

- novas tentativas em falhas temporárias do Gemini;
- formulário manual quando a interpretação automática não funciona;
- resposta local quando a redação final pelo Gemini falha;
- manutenção do registro CSV quando o Google Sheets está indisponível;
- validação do CSV antes da execução dos cálculos;
- mensagens de erro e status apresentadas na interface.

Dessa forma, as partes essenciais da aplicação continuam disponíveis mesmo quando integrações externas apresentam instabilidade.

6 Possíveis melhorias futuras

A solução atual atende ao fluxo proposto, mas pode ser ampliada para aumentar sua robustez, flexibilidade e capacidade de uso em cenários reais.

Entre as principais melhorias futuras estão:

- **Uso de uma plataforma de linguagem mais estável:**

Substituição da versão gratuita do Gemini por uma solução com maior disponibilidade, limites de uso mais previsíveis, melhor controle de custos e suporte adequado para aplicações em produção.

- **Expansão do chat para outros tipos de problema:**

Ampliação da interface para permitir perguntas mais abertas, consultas sucessivas e análises além do ranking e do relatório geral, tornando o chat capaz de lidar com diferentes necessidades de negócio.

- **Identificação automática do tipo de dataset:**

Análise das colunas disponíveis no arquivo para reconhecer automaticamente sua estrutura, adaptar as métricas calculadas e oferecer suporte a conjuntos de dados diferentes dos utilizados neste projeto.

- **Criação de regras específicas por contexto:**

Definição de métricas, alertas e critérios de decisão diferentes conforme o tipo de parceiro, campanha, produto ou objetivo analisado.

- **Maior segurança das integrações:**

Uso de autenticação mais robusta, controle de permissões, armazenamento seguro de credenciais, validação das requisições e proteção dos serviços externos utilizados pela aplicação.

- **Melhoria da validação dos dados:**

Inclusão de verificações mais detalhadas para valores inconsistentes, duplicidades, formatos inesperados, períodos incompletos e possíveis anomalias nos arquivos recebidos.

- **Histórico de conversas e análises:**

Armazenamento do contexto das interações para permitir perguntas de continuidade, comparação com análises anteriores e acompanhamento da evolução dos testes.

- **Banco de dados dedicado:**

Substituição do histórico em CSV e da integração simples com planilhas por um banco de dados estruturado, permitindo maior volume, controle, consulta e rastreabilidade.

- **Controle de acesso:**

Inclusão de autenticação de usuários e diferentes níveis de permissão para visualização, execução, aprovação e administração das análises.

- **Publicação em ambiente web:**

Disponibilização da aplicação em uma infraestrutura de nuvem, com monitoramento, logs, atualização contínua e acesso sem necessidade de execução local.

- **Testes automatizados:**

Criação de testes para validar preparação dos dados, cálculos, rankings, alertas, geração de relatórios e comportamento em situações de falha.

- **Monitoramento de desempenho e erros:**

Registro estruturado de falhas, tempo de execução, uso do modelo de linguagem e status das integrações para facilitar manutenção e melhoria contínua.

Essas mudanças permitiriam transformar o protótipo em uma solução mais geral, segura e adequada para uso contínuo em diferentes contextos de negócio.

Referências

[Beiro 2026]BEIRO, P. M. *Acompanhamento de Testes A/B de Cash-back*. 2026. Google Sheets. https://docs.google.com/spreadsheets/d/1XggGi-xdwQ1WcOGQejSepJ9CXXuR4eYWv3kUAE_TDHM/edit?usp=sharing. Acesso em: 16 jun. 2026.